МИНИСТЕРСТВО ОБРАЗОВАНИЯ СВЕРДЛОВСКОЙ ОБЛАСТИ
ГАПОУ СО «ЕКАТЕРИНБУРГСКИЙ МОНТАЖНЫЙ КОЛЛЕДЖ»

КУРСОВОЙ ПРОЕКТ

По дисциплине «Тестирование кода информационной системы»

ТЕМА: «Создание и тестирование АРМ фитнес-центра»

0	9	0	2	0	7	- КП -	7	0	2	7	И	П	-	З	4	-	2	З

ОЦЕНКА: _____
РУКОВОДИТЕЛЬ КУРСОВОГО ПРОЕКТА:
_____(_____)
СТУДЕНТ гр. ИП-311
Зырянов Алан Русланович (_____)

2025

№ стр.	Формат	Обозначение	Наименование	Кол. листов	№ экз.	Примеч.
1	А4	ЕМК 09.02.07. КП 7027ИП-34-23 ПЗ	Пояснительная записка	46	-	
2		ЕМК 09.02.07.КП 7027ИП-34-23 ИС	«Создание и тестирование АРМ фитнес-центра»	1		USB флеш-карта
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						
21						
22						
23						
24						
25						

Изм.	Лист	№ докум.	Подпись	Дата	ЕМК 09.02.07 КП 7027ИП-34-23		
Разраб.		Зырянов А. Р.			Ведомость курсового проекта	Лит.	Лист 2 / Листов 46
Провер.		Черноскутов Д. А.					
Реценз.						ГАПОУ СО «ЕМК» Группа ИП-311	
Н. Контр.							
Утверд.							

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 3
ВВЕДЕНИЕ 5
1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ 6
2. АНАЛИТИЧЕСКАЯ ЧАСТЬ 7
2.1. Цели, функции и задачи системы 7
2.2. Описание предметной области системы 8
2.3. Определение пользователей системы 9
2.4. Проектные решения по реализации внутренней архитектуры приложения 12
2.5. Проектные решения по реализации тестирования приложения 14
3. ПРОЕКТНАЯ ЧАСТЬ 18
3.1. ERD-модель информационной системы 18
3.2. Файловая структура приложения 22
3.3 Алгоритмы 24
3.4 Документация по работе с информационной системой 26
3.5. Отчет о тестировании проекта 34
4. ГЛОССАРИЙ 37
5. СПИСОК ИСТОЧНИКОВ 41
ЗАКЛЮЧЕНИЕ 43
ПРИЛОЖЕНИЕ А 44
ПРИЛОЖЕНИЕ Б 45

ВВЕДЕНИЕ

В настоящее время автоматизация деятельности организаций является одним из ключевых направлений развития информационных технологий. Особенно актуально использование информационных систем для фитнес-центров, работа которых связана с обработкой большого объёма данных о клиентах, тренировках, расписании занятий и персонале. Использование ручного учёта или разрозненных программных решений снижает эффективность работы сотрудников, увеличивает время обработки информации и повышает вероятность возникновения ошибок. В связи с этим разработка автоматизированного рабочего места фитнес-центра является актуальной задачей, направленной на повышение качества управления и обслуживания клиентов.

Целью курсового проекта является разработка и тестирование информационной системы для автоматизации деятельности фитнес-центра. Для достижения поставленной цели необходимо выполнить анализ предметной области, определить требования к системе, разработать архитектуру приложения, реализовать основные функции веб-приложения и API-бэкенда, а также провести тестирование разработанной системы.

Объектом исследования является процесс автоматизации деятельности фитнес-центра. Предметом исследования выступают методы и технологии проектирования, разработки и тестирования клиент-серверных информационных систем.

Разрабатываемая система предназначена для хранения и обработки данных пользователей, управления тренировками, расписанием и взаимодействием между клиентами, тренерами и администраторами. Использование веб-технологий и клиент-серверной архитектуры позволяет обеспечить централизованный доступ к системе и возможность дальнейшего расширения её функциональности.

Практическая значимость работы заключается в создании информационной системы, способной повысить эффективность работы фитнес-центра, упростить обработку данных и автоматизировать основные процессы организации.

Структура курсового проекта включает введение, аналитическую часть, проектную часть, раздел тестирования, заключение, список источников и приложения.

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Тема курсового проекта: «Создание и тестирование АРМ фитнес-центра»
Выдано Зырянову Алану Руслановичу студенту группы ИП-311/ИП-311к
Сроки выполнения:
1. разработка REST-сервиса с использованием фреймворка FastAPI : до 15.12.25г
2. разработка web-приложения на Django": до 1.05.26г
ВИД АВТОМАТИЗИРУЕМОЙ ДЕЯТЕЛЬНОСТИ
Приложение предназначено для клиентов фитнес-центра. Цель приложения – создание фитнес – расписания тренерами и контроль за соблюдением выполнения предписанных тренировок клиентами .
Предполагается, что в системе фитнес центра будут определены следующие роли пользователей: клиенты, тренеры, менеджеры, администраторы.
Регистрацию тренеров в АРМ выполняет администратор. Для того, чтобы зарегистрировать тренера нужно указать: ФИО, фото, пол, стаж работы, список достижений. Регистрацию новых клиентов клуба ведут менеджеры. Для того, чтобы зарегистрировать клиента нужно указать: ФИО, телефон, пароль, дату рождения, фото, пол. Постоянным клиентам менеджеры могут предоставлять скидки на тренировки.
Любой клиент после авторизации (авторизация происходит по номеру телефона и паролю) может выбрать себе тренера (если у него нет такового). В этом случае клиент видит список тренеров с именем, фото, полом, стажем работы и списком достижений. Клиент может отправить заявку любому из тренеров, написав при этом цель, которую он хочет достигнуть при тренировках.
Тренер после авторизации видит новые заявки от клиентов и их количество (если таковые имеются). Тренер может принять заявку или отклонить. В случае отказа, тренер должен указать причину. В случае подтверждения заявки тренер должен выставить план индивидуальных занятий для клиента. Выбрав из списка клиентов без плана тренировок, тренер видит цель клиента, его возраст и планирует даты тренировочного цикла. Для индивидуальных занятий тренер может выбрать упражнения, указывая при этом его вид (приседания, отжимания и т.д.), частоту выполнения (сколько раз в неделю), число подходов и число повторений в каждом подходе.
Клиент, отправивший заявку, но не получивший ответа, видит список своих заявок с результатами (в том числе с указанием причины при отказе) и количеством дней ожидания ответа. Получив план тренировок, клиент видит экран с 2 вкладками: план тренировок (датасписок упражнений) и сегодняшний перечень индивидуальных занятий. Для последней выводится список: вид упражнения, количество повторов и «флажок», позволяющий отметить выполнено ли упражнение. Несмотря на это, упражнение не будет засчитано системой до тех пор, пока клиент не укажет показатель своего пульса во время выполнения упражнения.
Сверху выводится сегодняшний прогресс (по количеству выполненных упражнений) в процентах с графическим отображением. Тренер также может посмотреть список своих текущих клиентов с указанием у каждого: процента выполнения всего цикла тренировок (зависит от длительности цикла) и процента выполненных упражнений (т.к. некоторые упражнения могут быть пропущены). По каждому клиенту выводится средний показатель пульса во время выполнения упражнений.
Приложение может выполнять генерацию следующих отчетов (Word/Excel):
- Для фитнес-тренеров (список клиентов, расписание тренировок);
- Для клиентов фитнес-центра (план тренировок на сегодняшний день, расписание тренировок на неделю)

2. АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1. Цели, функции и задачи системы

Актуальность разработки автоматизированного рабочего места фитнес-центра обусловлена необходимостью повышения эффективности управления внутренними процессами организации в условиях постоянного увеличения объёма данных и количества пользователей. В процессе работы фитнес-центра ежедневно выполняются операции, связанные с регистрацией клиентов, управлением расписанием тренировок, формированием тренировочных программ, обработкой заявок и контролем доступа пользователей к различным функциям системы. При использовании ручного учёта или разрозненных программных решений возрастает вероятность ошибок, усложняется контроль данных и увеличивается нагрузка на сотрудников фитнес-центра.

Основной целью разрабатываемой системы является создание единого программного комплекса, который позволит автоматизировать основные процессы фитнес-центра, обеспечить централизованное хранение информации и повысить удобство взаимодействия между клиентами, тренерами и администрацией. Предполагается, что система будет обеспечивать безопасную обработку данных, поддержку ролевой модели доступа и возможность дальнейшего расширения функциональности.

Информационная система будет включать несколько основных компонентов. Одним из компонентов станет API-бэкенд с поддержкой версионирования API (v1 и v2). API будет использоваться для обработки запросов, взаимодействия с базой данных, реализации бизнес-логики и поддержки механизмов аутентификации и авторизации пользователей. Для API предполагается использование автоматической документации Swagger/OpenAPI.

Вторым компонентом системы станет полнофункциональное веб-приложение, реализуемое на основе Django. Веб-приложение будет использовать шаблоны, формы и административную панель для взаимодействия с пользователями. Получение и обработка данных будут выполняться через REST API, что позволит разделить пользовательский интерфейс и серверную бизнес-логику.

Для хранения информации предполагается использование клиент-серверной базы данных PostgreSQL, обеспечивающей централизованное хранение данных пользователей, расписания тренировок, заявок и тренировочных программ.
К основным функциям системы относятся:

- регистрация и авторизация пользователей;
- управление ролями и правами доступа;
- хранение и обработка данных клиентов;
- создание тренировочных программ;
- управление расписанием тренировок;
- обработка заявок на занятия;
- взаимодействие между клиентами и тренерами;
- администрирование системы.

Для достижения поставленной цели необходимо решить следующие задачи:

- Выполнить анализ предметной области и определить требования к информационной системе фитнес-центра.

- Разработать архитектуру приложения и структуру взаимодействия между веб-приложением, API и базой данных.
- Реализовать механизмы аутентификации, авторизации и разграничения прав доступа.
- Разработать основные модули системы и REST API.
- Выполнить тестирование основных функций и компонентов приложения.

2.2. Описание предметной области системы

Предметная область проекта охватывает деятельность фитнес-центра в части управления клиентами, организации тренировочного процесса, формирования тренировочных программ и контроля взаимодействия между пользователями системы. Анализ предметной области выполняется не только как описание сущностей базы данных, но и как исследование основных процессов работы фитнес-центра, требующих автоматизации.

В основу проектирования системы предполагается положить последовательность основных этапов взаимодействия пользователей:

регистрация пользователя → выбор тренера → подача заявки на тренировку → формирование тренировочного плана → выполнение тренировок → отслеживание прогресса → формирование отчетности.

Именно данный процесс определяет состав сущностей и взаимосвязей внутри системы. Основная задача проектирования заключается в обеспечении непрерывности хранения и обработки данных на всех этапах работы пользователей с приложением.

С точки зрения информационных потоков в системе предполагается выделение следующих основных объектов предметной области:

- Клиент - персональные данные, контактная информация, история тренировок и показатели прогресса;
- Тренер - данные о специализации, расписании и закрепленных клиентах;
- заявка на тренировку - дата создания, статус заявки, выбранный тренер;
- Тренировочный план - набор упражнений, расписание занятий, параметры тренировок;
- Отчет о тренировках - информация о выполненных упражнениях и достигнутых результатах;
- Пользователь системы - учетные данные, роль и права доступа.

Выделение именно этих сущностей обусловлено спецификой работы фитнес-центра и необходимостью централизованного хранения информации о тренировочном процессе.

В предметной области предполагается использование нескольких ролей пользователей:

- Администратор - управление тренерами и учетными записями;
- Менеджер - работа с клиентской базой и заявками;
- Тренер - создание тренировочных программ и контроль прогресса клиентов;
- Клиент - просмотр тренировок, отметка выполненных упражнений и отслеживание результатов.

Такое разделение ролей позволит ограничить доступ пользователей только к тем функциям и данным, которые относятся к их обязанностям. Для реализации данного механизма предполагается использование ролевой модели доступа (RBAC).

Для формирования структуры проекта предполагается изучение существующих информационных систем фитнес-центров, CRM-систем и веб-приложений для управления тренировочным процессом. Также будут анализироваться процессы взаимодействия между сотрудниками фитнес-центра и клиентами, структура тренировочных программ и принципы хранения пользовательских данных.

Какие исходные данные используются и почему:

Для формирования структуры проекта предполагается использование нескольких источников информации, позволяющих приблизить разрабатываемую систему к реальным условиям работы фитнес-центра.

В качестве основных исходных данных рассматриваются:

1. Требования задания и методические рекомендации - для определения обязательного функционала системы, структуры модулей и состава основных сущностей базы данных.
2. Анализ процессов работы фитнес-центра - для определения сценариев взаимодействия между клиентами, тренерами, менеджерами и администраторами.
3. Анализ существующих веб-приложений и CRM-систем для фитнес-клубов - для изучения структуры тренировочных программ, системы заявок и организации клиентской базы.
4. Изучение принципов организации тренировочного процесса - для определения структуры тренировочных планов, расписаний и отчетов о прогрессе клиентов.

Ключевым критерием отбора исходных данных является их практическая значимость для работы системы. Каждый элемент данных должен участвовать либо в организации тренировочного процесса, либо в обеспечении корректного взаимодействия пользователей внутри системы.

Для тестирования предполагается использование данных, приближенных к реальным условиям эксплуатации. Каждая основная таблица базы данных должна содержать не менее 50 тестовых записей. Для генерации данных предполагается использование библиотеки Faker и подготовленных тестовых наборов.

Проблемы предметной области и обоснование автоматизации

При отсутствии единой информационной системы в работе фитнес-центра возникают типовые проблемы:

- разрозненное хранение информации о клиентах и тренировках;
- сложность отслеживания прогресса клиентов;
- необходимость повторного ручного ввода данных;
- ошибки при работе с расписанием и заявками;
- сложность взаимодействия между сотрудниками и клиентами;
- отсутствие централизованного контроля тренировочного процесса.

Автоматизация данных процессов позволит сократить количество рутинных операций и повысить эффективность работы фитнес-центра.

Реализация информационной системы должна обеспечить:

- централизованное хранение данных;
- единые сценарии обработки информации;
- разграничение прав доступа пользователей;
- взаимосвязь между клиентами, тренерами, тренировочными планами и заявками;
- повышение скорости обработки информации и качества обслуживания клиентов.

2.3. Определение пользователей системы

Целевой аудиторией системы являются клиенты фитнес-центра, тренеры, менеджеры и администраторы, так как именно эти категории пользователей ежедневно взаимодействуют с данными о тренировках, расписании и клиентской базе. Интерфейс системы должен быть адаптирован не под технических специалистов, а под пользователей, которым необходимо быстро выполнять рабочие операции и получать доступ к нужной информации без сложной навигации.

Основной сценарий использования системы строится по принципу минимального количества действий:
авторизация → выбор раздела → работа с тренировками или клиентами → сохранение изменений → просмотр результатов.

Навигация системы должна быть организована по логике работы пользователей, а не по внутренней структуре модулей приложения. Это позволит упростить взаимодействие с системой и сократить время выполнения основных операций.

Для системы предполагается использование ролевого интерфейса (role-based UI). Каждый пользователь будет видеть только те разделы и функции, которые относятся к его роли:
- клиент - тренировки, прогресс, заявки;
- тренер - клиенты, тренировочные планы, отчеты;
- менеджер - клиентская база и заявки;
- администратор - управление тренерами.

Такой подход позволит уменьшить вероятность ошибок и снизить нагрузку на пользователя за счет отображения только необходимого функционала.

При проектировании интерфейса предполагается использование единообразной структуры страниц: список данных → форма создания или редактирования → сохранение результата. Это позволит упростить освоение системы и сделать работу с приложением более понятной для пользователей с различным уровнем подготовки.

Особое внимание предполагается уделить скорости доступа к основным функциям системы. Для тренеров важным элементом интерфейса станет быстрый доступ к тренировочным планам, заявкам клиентов и информации о прогрессе тренировок. Для менеджеров приоритетом будет удобная работа с клиентской базой и расписанием.

Разграничение доступа предполагается реализовать с использованием ролевой модели RBAC и JWT-аутентификации. Каждому пользователю будет назначаться определенный набор прав доступа в зависимости от его роли в системе.

В рамках проектирования предполагается разработка UML-диаграммы вариантов использования (Use Case Diagram). На диаграмме будут отражены основные участники системы (рис. 1):

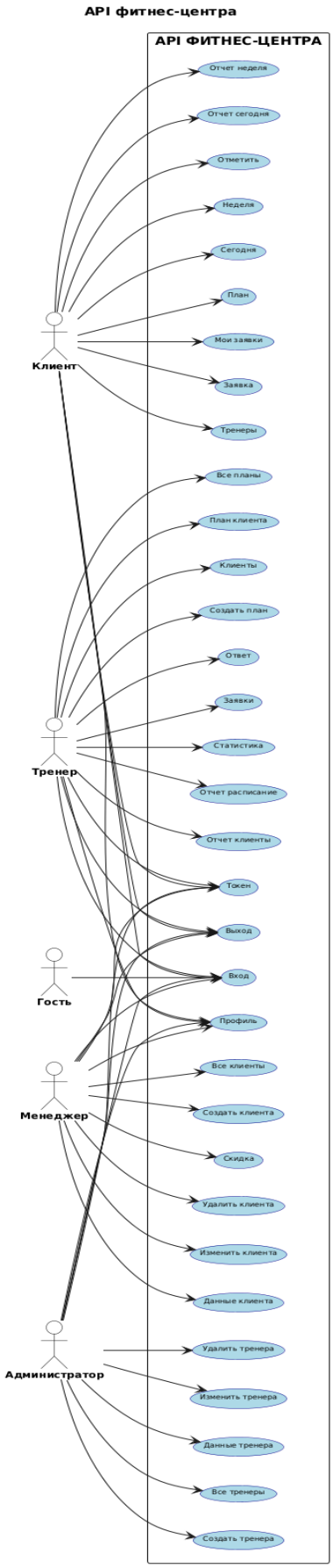


Рисунок 1, Диаграмма прецедентов

2.4. Проектные решения по реализации внутренней архитектуры приложения

В проекте планируется использование двухслойной архитектуры: Django применяется как web-слой пользовательского взаимодействия, а FastAPI - как сервисный слой бизнес-логики и REST API. Такое решение позволяет разделить ответственность между интерфейсной частью и обработкой данных: web-слой отвечает за отображение страниц, формы, маршрутизацию и взаимодействие с пользователем, а API-слой - за обработку бизнес-операций, проверку ролей, валидацию данных и работу с базой данных.

Взаимодействие компонентов системы организуется по следующей схеме:
пользователь → Django web-приложение → FastAPI REST API → PostgreSQL.

Внутренняя архитектура проекта строится на основе модульного подхода. Функциональность системы разделяется на отдельные уровни и модули, каждый из которых отвечает за строго определенную область задач. Такой подход позволяет обеспечить слабую связанность компонентов, высокую читаемость кода и возможность независимого развития отдельных частей приложения.

Использованный стек технологий

- Python 3.11+ - основной язык разработки всех компонентов системы;
- Django - серверный web-интерфейс, шаблоны, формы, маршруты и административная панель;
- FastAPI - REST API, обработка запросов, Redoc/Swagger-документация;
- PostgreSQL - клиент-серверная база данных;
- SQLModel + Pydantic - модели данных, ORM, типизация и валидация запросов;
- JWT (python-jose / bcrypt) - аутентификация и разграничение доступа;
- Requests - взаимодействие Django-приложения с FastAPI;
- Selenium + Pytest - автоматизированное тестирование пользовательских сценариев

Архитектурные уровни приложения

1. Presentation Layer (слой представления)

Слой представления отвечает за взаимодействие пользователя с системой и обработку HTTP-запросов.

В состав уровня входят:

- FastAPI-роутеры;
- Django views и HTML-шаблоны;
- формы ввода данных;
- маршруты приложения;
- Pydantic-схемы для проверки входящих данных.

На данном уровне выполняются:

- обработка пользовательских запросов;
- передача данных в сервисный слой;
- возврат HTTP-ответов;
- сериализация и валидация данных.

2. Business Logic Layer (слой бизнес-логики)

Слой бизнес-логики отвечает за реализацию основных правил работы системы и обработку пользовательских сценариев.

В данном слое располагаются:

- контроллеры;

- сервисы;
- механизмы проверки бизнес-правил;
- обработка ролевого доступа.

Сервисный слой отвечает за:
- обработку заявок клиентов;
- создание тренировочных планов;
- управление упражнениями;
- расчет прогресса;
- проверку прав доступа пользователей.

Разделение бизнес-логики и HTTP-обработки позволяет упростить тестирование и сопровождение проекта.

3. Data Access Layer (слой доступа к данным)

Слой доступа к данным отвечает за взаимодействие приложения с базой данных PostgreSQL.

В состав уровня входят:
- ORM-модели SQLModel;
- асинхронные сессии базы данных;
- репозитории для работы с сущностями.

Слой используются для:
- получения данных;
- создания записей;
- обновления информации;
- удаления объектов;
- фильтрации и поиска.

Такое разделение позволяет изолировать SQL-запросы от бизнес-логики приложения.

4. Database Layer (слой базы данных)

На уровне базы данных осуществляется хранение информации обо всех сущностях системы.

В PostgreSQL предполагается хранение:

учетных записей пользователей;
- клиентов фитнес-центра;
- тренеров;
- заявок на тренировки;
- тренировочных планов;
- упражнений;
- данных о прогрессе клиентов.

Структура базы данных проектируется с учетом связей между сущностями и требований к целостности данных.

Выбор технологий выполнен по критериям: скорость разработки, читаемость архитектуры, тестопригодность, удобство сопровождения и возможность дальнейшего расширения проекта.

Использование Python как единого языка разработки снижает сложность сопровождения системы и упрощает взаимодействие между компонентами.

Django и FastAPI позволяют разделить пользовательский интерфейс и бизнес-логику приложения, сохранив удобство серверного web-интерфейса и гибкость API.

SQLModel и Pydantic обеспечивают строгую типизацию и валидацию данных, что уменьшает количество ошибок обработки запросов.

PostgreSQL обеспечивает надежное хранение данных и поддержку многопользовательской работы.
JWT и ролевая модель доступа позволяют реализовать разграничение прав пользователей и защиту API-маршрутов.
Selenium и Pytest позволяют выполнять тестирование как пользовательских сценариев, так и API-контрактов backend-части приложения.
Сравнение с альтернативами
Альтернатива 1: монолитное приложение только на Django
Преимущества:
- более простая начальная реализация;
- меньшее количество компонентов системы.

Недостатки:
- высокая связанность модулей;
- сложность изолированного тестирования бизнес-логики;
- усложнение масштабирования API.

Выбранная архитектура с разделением Django и FastAPI обеспечивает более удобное развитие проекта и лучшую модульность системы.
Альтернатива 2: SPA-фронтенд (React/Vue) + API
Преимущества:
- высокая гибкость пользовательского интерфейса;
- возможность создания полноценного SPA-приложения.

Недостатки:
- увеличение сложности проекта;
- необходимость отдельной разработки frontend-части;
- более высокий порог входа для учебного проекта.

Использование Django как web-слоя позволяет быстрее реализовать интерфейс системы и сосредоточиться на проектировании backend-архитектуры и API.

2.5. Проектные решения по реализации тестирования приложения
Тестирование веб-приложения фитнес-центра рассматривается как отдельный этап контроля качества, направленный на подтверждение корректности работы пользовательских сценариев, API-маршрутов и механизмов разграничения доступа.
Проверке подлежат как отдельные компоненты системы, так и взаимодействие между Django-интерфейсом и FastAPI-бэкендом.

Основными целями тестирования являются:
- проверка корректности работы пользовательских сценариев;
- подтверждение работоспособности REST API;
- проверка обработки JWT-токенов и ролевой модели доступа;
- выявление ошибок валидации входных данных;
- контроль корректности HTTP-статусов и сообщений об ошибках;
- проверка взаимодействия веб-интерфейса и API;
- обеспечение стабильности системы при изменении функциональности.

Тестирование охватывает следующие функциональные области системы:
- регистрация и аутентификация пользователей;
- авторизация по ролям (client, trainer, manager, admin);
- CRUD-операции с клиентами и тренерами;
- создание и обработка заявок;

- работа с тренировочными планами;
- просмотр прогресса и расписания;
- генерация отчетов;
- обработка ошибок и невалидных запросов;
- защита маршрутов и API-эндпоинтов.

Для проекта используется комбинированная стратегия тестирования, включающая:
- функциональное тестирование;
- интеграционное тестирование;
- UI-тестирование;
- API-тестирование;
- негативное тестирование;
- регрессионное тестирование.

Такой подход позволяет проверять систему как на уровне отдельных модулей, так и на уровне полного пользовательского сценария.

Все тесты должны быть:
- воспроизводимыми;
- независимыми друг от друга;
- пригодными для автоматизированного запуска;
- понятными по структуре;
- изолированными от пользовательских данных.

Для подготовки тестовых данных используются:
- тестовая база данных;
- заранее подготовленные учетные записи;
- автоматически сгенерированные данные;
- фикстуры и mock-объекты.

Таблица 1. План тестирования

Test Case ID	Объект исследования	Стратегия тестирования	Критерии начала	Критерии окончания	Последовательность проведения
TC_1	Регистрация пользователя и авторизация (Req_1, Req_2)	UI, API, функциональное, интеграционное, негативное	Доступны форма регистрации, endpoint авторизации и тестовые роли	Пользователь успешно регистрируется, валидный JWT предоставляет доступ, ошибки обрабатываются корректно	Подготовка тестовых данных → регистрация пользователя → получение JWT-токена → проверка доступа к защищенным маршрутам
TC_2	CRUD клиентов (Req_3)	UI + API, функциональное, регрессионное	Подготовлены тестовые пользователи и база данных	Операции Create/Read/Update/Delete выполняются корректно	Создание клиента → просмотр списка → редактирование → удаление

TC_3	CRUD тренеров (Req_4)	UI + API, функциональное, интеграционное	Доступна роль администратора	Управление тренерами выполняется без ошибок и нарушений доступа	Добавление тренера → просмотр данных → редактирование → удаление
TC_4	Работа с заявками на тренировки (Req_5)	UI + API, интеграционное, функциональное	Созданы тестовые клиенты и тренеры	Заявки создаются и изменяют статус корректно	Создание заявки клиентом → обработка заявки тренером → проверка статуса
TC_5	Тренировочные планы и расписание (Req_6, Req_7)	UI + API, функциональное, регрессионное	Активна роль trainer, доступны тестовые данные	Планы тренировок и расписание отображаются корректно	Создание тренировочного плана → просмотр расписания → проверка отображения данных
TC_6	Генерация отчетов (Req_8)	UI, интеграционное	Доступны тестовые данные клиентов и тренировок	Отчеты формируются без ошибок сервера	Формирование DOCX/XLSX-отчетов → проверка скачивания и содержимого
TC_7	Ролевой доступ (Req_9)	UI + API, безопасность, негативное	Созданы роли client/trainer/manager/admin	Недопустимые действия блокируются системой	Авторизация под разными ролями → проверка доступа к разделам и API
TC_8	Ошибки API и обработка невалидных запросов (Req_10)	API, негативное	Доступен тестовый API-клиент	API возвращает корректные HTTP-коды и сообщения ошибок	Отправка невалидных запросов → проверка ответов 4xx/5xx

Таблица 2. Traceability Matrix

Requirements ID	TC_1	TC_2	TC_3	TC_4	TC_5	TC_6	TC_7	TC_8
Req_1	+							
Req_2	+						+	+
Req_3		+					+	
Req_4			+				+	
Req_5				+				
Req_6					+			

Req_7					+			
Req_8						+		
Req_9							+	
Req_10								+

3. ПРОЕКТНАЯ ЧАСТЬ

3.1. ERD-модель информационной системы

Физическая модель базы данных представлена в виде ERD-диаграммы (Entity-Relationship Diagram), реализованной средствами СУБД PostgreSQL. Диаграмма отражает структуру таблиц, их атрибуты, первичные и внешние ключи, а также связи между сущностями.

ERD-диаграмма базы данных «АРМ фитнес-центра» представлена на рисунке 2.

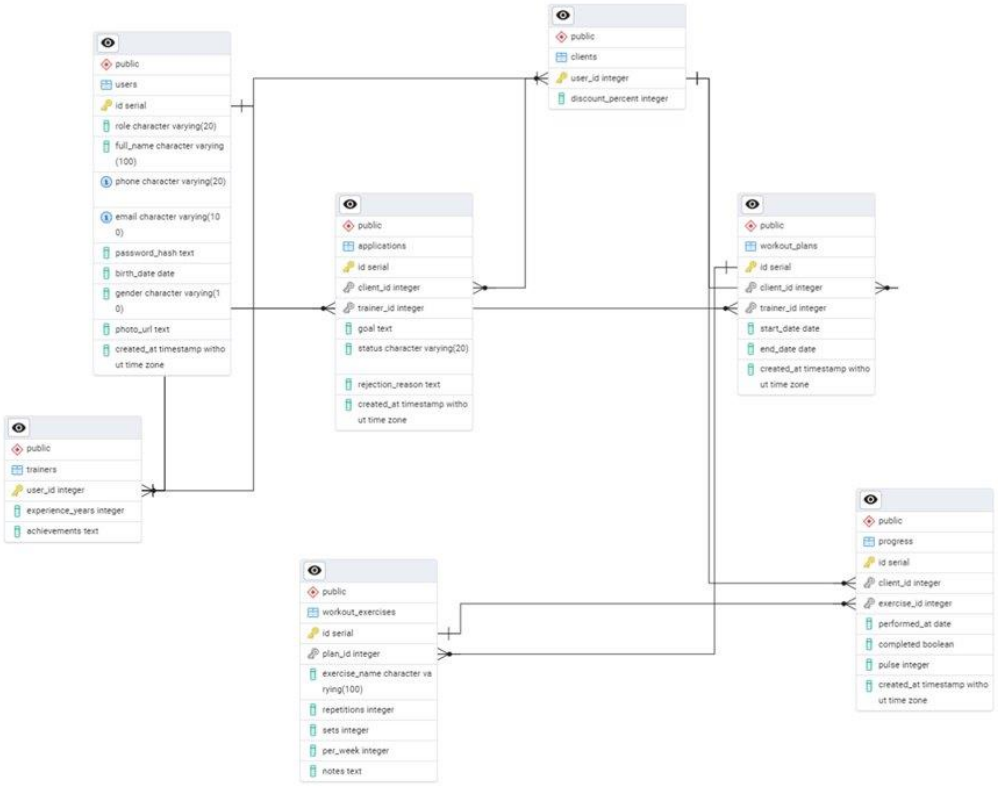


Рисунок 2, ERD-диаграмма базы данных

База данных информационной системы включает 7 взаимосвязанных таблиц: users, clients, trainers, applications, workout_plans, workout_exercises, progress. Таблица users является центральной и хранит общие данные для всех пользователей системы. Таблицы clients и trainers расширяют данные пользователей в соответствии с их ролью. Таблица applications фиксирует заявки клиентов на тренировки. Таблица workout_plans содержит планы тренировок, привязанные к клиенту и тренеру. Таблица workout_exercises описывает конкретные упражнения в рамках плана. Таблица progress отслеживает прогресс выполнения упражнений клиентами.

Словарь данных сформирован для однозначного понимания структуры и назначения каждого поля, описанного в сущностях спроектированной базы данных. Ниже представлены словари данных для каждой из 7 таблиц базы данных.

Таблица 3 - Таблица users (Пользователи)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с	Уникальный

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
				шагом 1	идентификатор пользователя
	role	VARCHAR(20)	N		Роль пользователя (client, trainer, admin)
	full_name	VARCHAR(100)	N		Полное имя пользователя
	phone	VARCHAR(20)	Y		Номер телефона
	email	VARCHAR(100)	N	Уникальное	Адрес электронной почты
	password_hash	TEXT	N		Хэш пароля пользователя
	birth_date	DATE	Y		Дата рождения
	gender	VARCHAR(10)	Y		Пол пользователя
	photo_url	TEXT	Y		URL фотографии профиля
	created_at	TIMESTAMP	Y	DEFAULT NOW()	Дата и время регистрации

Таблица 4 -Таблица clients (Клиенты)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK, FK	user_id	INTEGER	N		ID пользователя (ссылка на users.id)
	discount_percent	INTEGER	Y	DEFAULT 0	Процент скидки клиента

Таблица 5 -Таблица trainers (Тренеры)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK, FK	user_id	INTEGER	N		ID пользователя (ссылка на users.id)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
	experience_years	INTEGER	Y		Количество лет опыта тренера
	achievements	TEXT	Y		Описание достижений тренера

Таблица 6 -Таблица applications (Заявки)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор заявки
FK	client_id	INTEGER	N		ID клиента (ссылка на clients.user_id)
FK	trainer_id	INTEGER	Y		ID тренера (ссылка на trainers.user_id)
	goal	TEXT	Y		Цель клиента
	status	VARCHAR(20)	N	DEFAULT 'pending'	Статус заявки (pending, approved, rejected)
	rejection_reason	TEXT	Y		Причина отклонения заявки
	created_at	TIMESTAMP	Y	DEFAULT NOW()	Дата и время создания заявки

Таблица 7 -Таблица workout_plans (Планы тренировок)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор плана тренировок
FK	client_id	INTEGER	N		ID клиента (ссылка на clients.user_id)
FK	trainer_id	INTEGER	N		ID тренера (ссылка на

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
					trainers.user_id)
	start_date	DATE	Y		Дата начала плана
	end_date	DATE	Y		Дата окончания плана
	created_at	TIMESTAMP	Y	DEFAULT NOW()	Дата и время создания плана

Таблица 8 - Таблица workout_exercises (Упражнения плана)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор упражнения
FK	plan_id	INTEGER	N		ID плана тренировок (ссылка на workout_plans.id)
	exercise_name	VARCHAR(100)	N		Название упражнения
	repetitions	INTEGER	Y		Количество повторений
	sets	INTEGER	Y		Количество подходов
	per_week	INTEGER	Y		Количество выполнений в неделю
	notes	TEXT	Y		Дополнительные заметки к упражнению

Таблица 9 - Таблица progress (Прогресс)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор записи прогресса
FK	client_id	INTEGER	N		ID клиента (ссылка на clients.user_id)
FK	exercise_id	INTEGER	N		ID упражнения (ссылка на

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
					workout_exercises.id)
	performed_at	DATE	Y		Дата выполнения упражнения
	completed	BOOLEAN	Y	DEFAULT FALSE	Признак выполнения упражнения
	pulse	INTEGER	Y		Пульс после выполнения
	created_at	TIMESTAMP	Y	DEFAULT NOW()	Дата и время создания записи

Такая схема соответствует 3-й нормальной форме, исключает избыточность данных и обеспечивает целостность через внешние ключи.

3.2. Файловая структура приложения

Файловая структура проекта организована в соответствии с принципами модульности, разделения ответственности и масштабируемости. Приложение разделено на две независимые части: серверную часть на FastAPI и клиентское веб-приложение на Django. Такое разделение позволяет независимо развивать API и пользовательский интерфейс, упрощает тестирование и сопровождение системы.

3.2.1 Структура FastAPI-приложения

```
fintes_zyranov2/                # корневая директория FastAPI-приложения
├── app/
│   ├── api/
│   │   └── routes/             # роутеры API
│   │       ├── admin.py          # маршруты администратора
│   │       ├── clients.py        # маршруты клиентов
│   │       ├── managers.py       # маршруты менеджеров
│   │       ├── profile.py        # маршруты профиля пользователя
│   │       └── trainers.py       # маршруты тренеров
│   │
│   ├── controllers/              # бизнес-логика приложения
│   ├── core/                     # конфигурация и безопасность
│   ├── db/                       # подключение к базе данных
│   ├── models/                   # ORM-модели SQLAlchemy
│   ├── schemas/                  # Pydantic-схемы
│   ├── utils/                    # вспомогательные функции
│   │
│   ├── auth.py                   # JWT-аутентификация и авторизация
│   ├── dependencies.py           # зависимости FastAPI
│   └── main.py                   # точка входа приложения
│
├── tests/                        # автоматизированные тесты
├── .env                          # переменные окружения
```

```
├── alembic.ini                # конфигурация миграций Alembic
└── requirements.txt           # зависимости проекта
```

Дополнительное пояснение назначения ключевых папок:

- Папка app/api/routes содержит маршруты FastAPI. Каждый файл соответствует отдельному модулю системы и отвечает за обработку HTTP-запросов.
- Папка controllers содержит бизнес-логику приложения: обработку данных, работу с моделями и проверку прав доступа.
- Папка core включает конфигурацию проекта, параметры безопасности и настройки JWT-аутентификации.
- Папка db отвечает за подключение к базе данных и создание ORM-сессий.
- Папка models содержит ORM-модели базы данных.
- Папка schemas содержит Pydantic-схемы для автоматической валидации данных.
- Папка utils включает вспомогательные функции и переиспользуемые утилиты.

3.2.2 Структура Django-приложения

```
fitness_django/            # корневая директория Django-приложения
├── app/
│   ├── assets/            # CSS, JS, изображения
│   ├── controllers/       # контроллеры взаимодействия с FastAPI
│   ├── migrations/        # миграции Django
│   ├── templates/         # HTML-шаблоны
│   │
│   ├── __init__.py
│   ├── admin.py           # регистрация моделей в admin-panel
│   ├── apps.py            # конфигурация Django-приложения
│   ├── models.py          # Django-модели
│   ├── tests.py           # тесты Django
│   ├── urls.py            # маршрутизация приложения
│   └── views.py           # представления Django
│
├── project/
│   ├── __init__.py
│   ├── asgi.py            # ASGI-конфигурация
│   ├── settings.py        # настройки проекта
│   ├── urls.py            # глобальная маршрутизация
│   └── wsgi.py            # WSGI-конфигурация
│
├── tests/
│   ├── integration/       # интеграционные тесты
│   ├── ui/                # UI-тестирование
│   ├── unit/              # модульные тесты
│   ├── __init__.py
│   └── conftest.py        # pytest-фикстуры
│
├── db.sqlite3             # база данных SQLite
├── manage.py              # точка входа Django
├── pytest.ini             # настройки pytest
└── requirements.txt       # зависимости проекта
```

Подобная организация структуры обеспечивает независимую разработку отдельных модулей системы, а использование централизованных контроллеров облегчает адаптацию приложения при изменении серверной части или переходе на другой API.

3.3 Алгоритмы

В данном разделе описаны ключевые алгоритмы, реализованные в информационной системе фитнес-центра. Каждый алгоритм представлен словесным описанием последовательности действий, а его графическое отображение в виде блок-схемы вынесено в Приложение Б.

3.3.1 Алгоритм авторизации и выдачи JWT (FastAPI)

Алгоритм обеспечивает аутентификацию пользователя по номеру телефона (или email) и паролю с последующей выдачей пары JWT-токенов (access и refresh). Реализован в контроллере AuthController. (см. Приложение Б, рисунок 13)

Последовательность действий:

1. Клиент (веб-приложение Django) отправляет POST-запрос на эндпоинт /api/v1/auth/login с полями username (номер телефона) и password.
2. Контроллер выполняет поиск пользователя в таблице users по полю phone:
3. Если пользователь не найден – возвращается статус 401 Unauthorized с сообщением «Неверный телефон или пароль».
4. Если найден – с помощью bcrypt проверяется соответствие переданного пароля хэшу password_hash в базе данных.
5. При несовпадении пароля – возвращается 401.
6. При успешной проверке генерируются:

access_token (срок жизни 30 минут, содержит sub = user_id и role);
refresh_token (срок жизни 24 часа).

7. Токены упаковываются в JSON и возвращаются клиенту.
8. Клиент сохраняет токены (в Django – в сессии, в API-клиенте – в localStorage или куках).
9. Во всех последующих запросах к защищённым эндпоинтам клиент передаёт access_token в заголовке Authorization: Bearer <token>.
10. Зависимость get_current_user в FastAPI декодирует токен, извлекает user_id, загружает объект пользователя из БД и проверяет его роль. При истекшем токене возвращается 401.

3.3.2 Алгоритм обработки заявки клиента на тренировку

Алгоритм описывает процесс создания заявки клиентом, её просмотра тренером и последующего принятия или отклонения с указанием причины. Реализован в контроллерах ApplicationsController и TrainersController. (см. Приложение Б, рисунок 14)

Последовательность действий:

1. Клиент авторизуется в системе (роль client).
2. Переходит в раздел «Выбор тренера».
3. Видит список тренеров с ФИО, фото, стажем и достижениями.
4. Выбирает тренера, нажимает «Отправить заявку».
5. Заполняет поле «Цель тренировок» (обязательное).
6. Отправляет POST-запрос на /api/v1/applications/ с полями trainer_id, goal.

Сервер создаёт запись в таблице applications со статусом pending (в ожидании), фиксирует created_at.

7. Возвращает клиенту сообщение об успешной отправке.

8. Тренер видит в своём кабинете количество новых заявок (значок на боковой панели).
9. Переходит в раздел «Новые заявки».
10. Система загружает список заявок со статусом pending, относящихся к текущему тренеру. По каждой заявке тренер может:
    - Принять – тогда статус меняется на approved, и тренер переходит к формированию тренировочного плана (алгоритм 3.3.3).
    - Отклонить – система запрашивает причину отказа (обязательное поле). После заполнения статус становится rejected, причина сохраняется в поле rejection_reason.
11. Клиент видит в разделе «Мои заявки» статус каждой своей заявки.
    - Если заявка отклонена – отображается причина отказа.
    - Если заявка принята – клиент получает доступ к тренировочному плану (после его создания тренером).

3.3.3 Алгоритм формирования тренировочного плана тренером

Алгоритм позволяет тренеру после принятия заявки создать индивидуальный план тренировок для клиента, включающий упражнения с заданием количества подходов, повторений и частоты выполнения.

Последовательность действий:

1. Тренер открывает карточку клиента (после принятия заявки или из списка «Мои клиенты»).
2. Нажимает кнопку «Создать тренировочный план».
3. Система открывает форму, в которой необходимо указать:
4. Даты начала и окончания плана (период действия).
5. После сохранения плана появляется возможность добавлять упражнения.
6. Для каждого упражнения тренер заполняет:
    - Название (например, «Приседания», «Отжимания»);
    - Количество повторений (repetitions);
    - Количество подходов (sets);
    - Частоту выполнения в неделю (per_week);
    - Дополнительные заметки (notes).
7. При отправке формы POST-запрос на /api/v1/workout-plans/{plan_id}/exercises/ создаёт запись в таблице workout_exercises, привязанную к плану.
8. Тренер может добавить несколько упражнений. План считается активным, когда добавлено хотя бы одно упражнение.
9. Клиент получает доступ к своему плану тренировок (вкладки «План тренировок» и «Сегодняшние упражнения»).
10. Система автоматически разбивает упражнения по дням согласно частоте per_week и датам плана (логика распределения реализована в сервисном слое).

3.3.4 Алгоритм отслеживания прогресса выполнения упражнений

Алгоритм позволяет клиенту отмечать выполнение упражнений, фиксировать пульс и автоматически рассчитывает процент выполнения как за день, так и за весь цикл тренировок.

Последовательность действий:

1. Клиент авторизуется и переходит на вкладку «Сегодняшние упражнения».
2. Система определяет текущую дату, загружает тренировочный план клиента и выбирает упражнения, которые должны выполняться сегодня (на основе поля per_week и логики распределения). Для каждого упражнения отображается:
    - название, количество подходов и повторений;

- чекбокс «Выполнено»;
- поле для ввода пульса (обязательно для засчитывания упражнения).
- Клиент выполняет упражнение, ставит отметку в чекбокс и вводит значение пульса.

3. При нажатии кнопки «Сохранить прогресс» отправляется PATCH-запрос на /api/v1/progress/ с параметрами: exercise_id, completed = true, pulse_value.
4. Сервер:
   - проверяет, что упражнение принадлежит плану клиента и соответствует текущей дате;
   - сохраняет запись в таблице progress (если запись за сегодня уже существует – обновляет её);
   - пересчитывает сегодняшний прогресс (процент выполненных упражнений от запланированных на день) и общий прогресс цикла (отношение количества уникальных дат, когда были выполнены все запланированные на день упражнения, к общей длительности плана в днях).
5. Обновлённые значения возвращаются клиенту.

3.4 Документация по работе с информационной системой

3.4.1. Документация по работе с API

API версионируется через префикс пути: /api/v1/ (базовая версия). В ходе тестирования дефектов, требующих изменений, не выявлено, поэтому версия v2 не создавалась. При необходимости расширения функциональности в будущем может быть введена версия v2 с сохранением обратной совместимости.

Версионирование:

v1 – первоначальная реализация, включающая полную поддержку пагинации, фильтрации, ролевой модели (client, trainer, manager, admin), JWT-аутентификацию и все требуемые эндпоинты.

Поддержка стандартных HTTP-методов

Для основных сущностей (клиенты, тренеры, заявки, тренировочные планы) реализованы стандартные HTTP-методы:

- Метод GET с эндпоинтом /api/v1/clients/trainers с параметрами пагинации ?page=1&size=10 возвращает список объектов.
- Метод GET с эндпоинтом /api/v1/managers/clients/{id} позволяет просмотреть один объект по его идентификатору.
- Создание нового объекта выполняется методом POST на эндпоинт /api/v1/clients/applications.
- Полное обновление объекта осуществляется методом PUT на /api/v1/managers/clients/{id}.
- Частичное обновление (например, скидки клиента) реализовано через метод PATCH на эндпоинте /api/v1/managers/clients/{id}/discount.
- Удаление объекта (деактивация) производится методом DELETE на /api/v1/managers/clients/{id}.

Аутентификация и авторизация.

Используется JWT (Bearer). После успешного входа сервер возвращает access_token (время жизни 60 минут) и refresh_token.

Токен передаётся в заголовке: Authorization: Bearer <access_token>.

В Swagger UI реализована кнопка «Authorize» для ввода токена.

Защита эндпоинтов реализована через зависимости FastAPI:

- get_current_user() – проверка валидности токена;

- role_required(["client"]), role_required(["trainer"]) и т.д. – доступ только для определённых ролей.

Автоматическая генерация Swagger UI и ReDoc документации для версии:
Swagger UI: http://localhost:8001/docs
ReDoc: http://localhost:8001/redoc
- Документация включает:
- полное описание всех эндпоинтов;
- модели и Pydantic-схемы запросов/ответов;
- статус-коды и примеры запросов;
- схему безопасности (JWT Bearer).

Особенности реализации в v1:
- На всех маршрутах, возвращающих списки, реализована пагинация через параметры page и size (по умолчанию 10 элементов на странице).
- Реализована фильтрация заявок по статусу (?status=pending/approved/rejected).
- Для роли client доступны: выбор тренера, создание заявки, просмотр тренировочного плана, ежедневное выполнение упражнений с обязательной фиксацией пульса, выгрузка отчётов (Word/Excel).
- Для роли trainer доступны: просмотр и обработка заявок (принятие/отклонение с указанием причины), создание тренировочных планов с упражнениями, просмотр списка клиентов и их прогресса, выгрузка отчётов.
- Для роли manager доступно полное CRUD-управление клиентами (создание, чтение, обновление, удаление, управление скидками).
- Для роли admin доступно полное CRUD-управление тренерами.

3.5.2 Документация по работе с веб-приложением

Веб-приложение на Django работает как прокси к FastAPI: все данные читаются и сохраняются через контроллеры, отправляющие HTTP-запросы к API. Сессии Django хранят access_token и роль пользователя. При получении от API статуса 401 сессия сбрасывается, и пользователь перенаправляется на страницу входа. Базовый URL веб-приложения: http://localhost:8000.

Общие элементы интерфейса:

Шапка сайта содержит навигационные ссылки, которые зависят от роли авторизованного пользователя. Для всех пользователей доступна кнопка «Выход» и переход в профиль.

Таблица 10, Тестовые авторизационные данные:

Роль	Логин	Пароль
Администратор	+79169130500	admin123
Тренер	+79161137611	trainer123
Менеджер	+7916361717	manager123
Клиент	+79162222222	client123

1. Авторизация

На главной странице GET / отображается форма входа (логин) – рисунок 3. Пользователь вводит номер телефона и пароль. При отправке POST-запроса данные передаются на POST /api/v1/auth/login. В случае успеха access-токен сохраняется в сессии, и выполняется редирект на дашборд, соответствующий роли пользователя (администратор → /panel/, менеджер → /manager/, тренер → /trainer/, клиент → /client/).

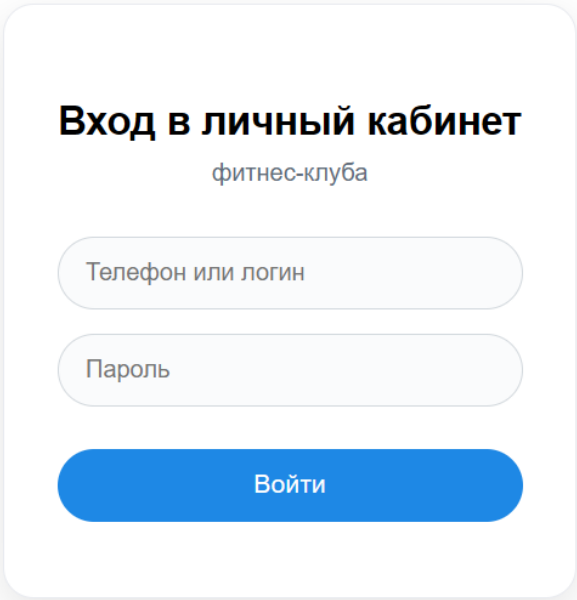


Рисунок 3, Форма авторизации

2. Управление тренерами (администратор)

Администратор входит в раздел /panel/, где видит список всех тренеров в виде карточек с возможностью поиска по ID (рисунок 4). Для добавления нового тренера необходимо нажать на кнопку добавить тренера и заполнить форму (ФИО, телефон, пароль, email, дата рождения, пол, стаж, достижения) и отправить POST-запрос. Для просмотра, редактирования и удаления существующего тренера используется достаточно нажать на карточку тренера.

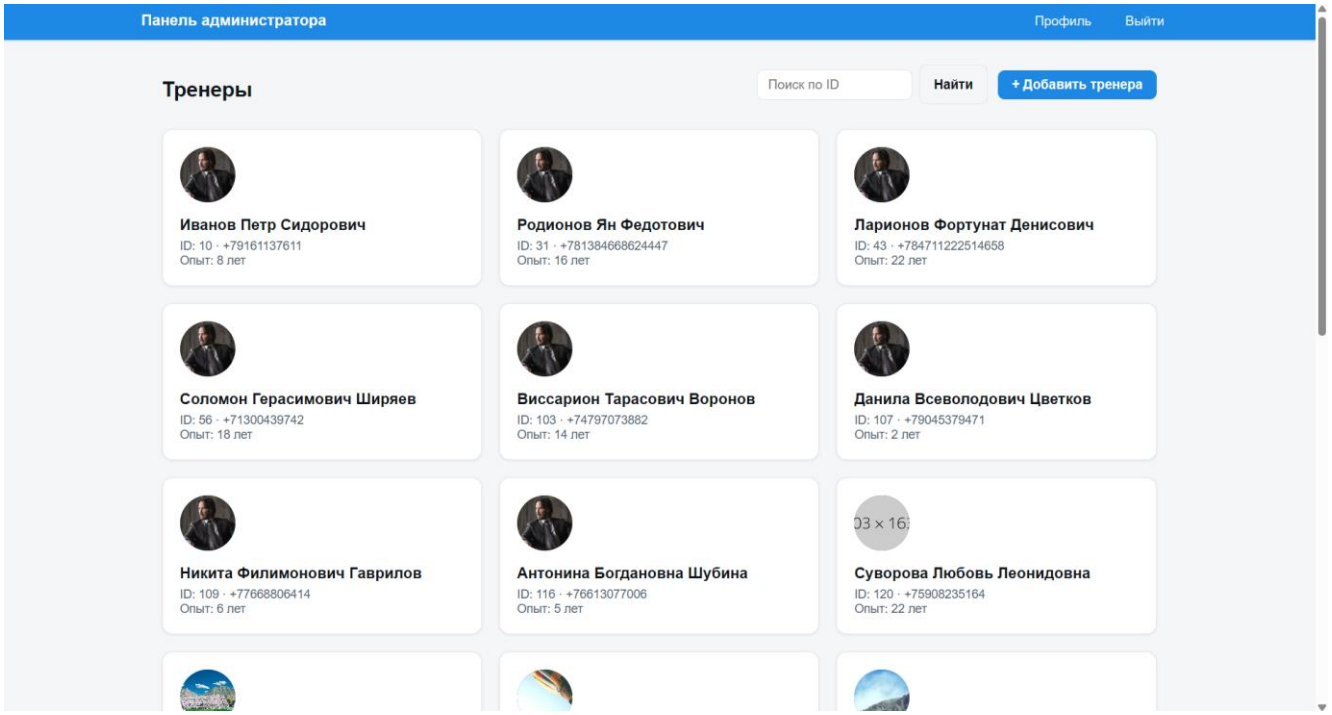


Рисунок 4, Список тренеров в панели администратора

3. Управление клиентами (менеджер)

Менеджер в разделе /manager/ видит список клиентов (карточки, поиск). Для добавление клиента нужно нажать на кнопочку и заполнить форму (указываются ФИО, телефон, пароль, email, дата рождения, пол, фото, скидка). Просмотр, редактирование и удаление при нажатии на клиента, также можно изменить и скидку клиента.

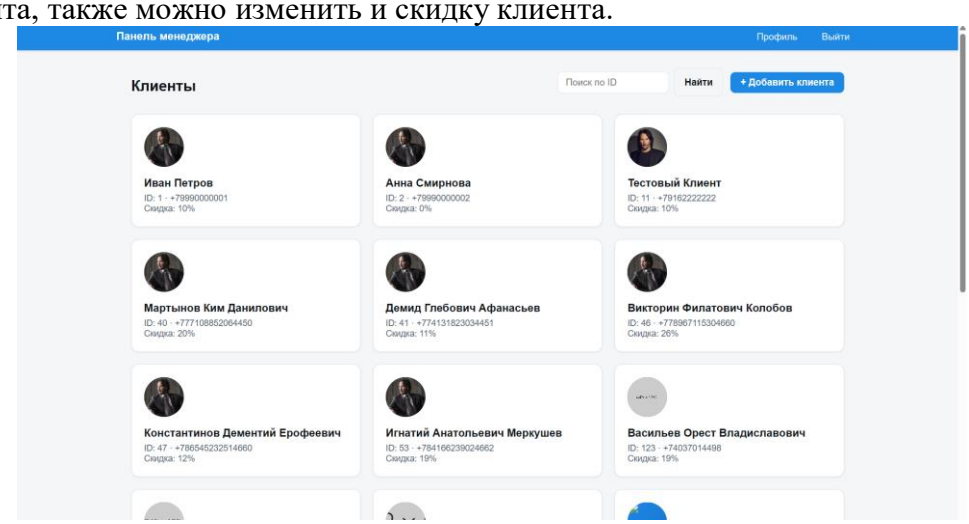


Рисунок 5, Список клиентов в панели менеджера

4. Работа тренера с заявками и планами

Тренер после входа попадает на главную панель /trainer/. В разделе заявок (/trainer/applications/) отображается список с фильтром по статусу (?status=pending, accepted, rejected) и пагинацией.

В разделе /trainer/clients/ тренер видит список своих текущих клиентов. Для создания тренировочного плана перейдите на GET /trainer/plans/add/ – откроется форма выбора клиента,

дат начала и окончания плана, а также добавления упражнений (название, повторения, подходы, частота в неделю, заметки). После отправки POST-запроса план сохраняется и становится доступен клиенту. Просмотр плана конкретного клиента – GET /trainer/plans/client/<client_id>/.

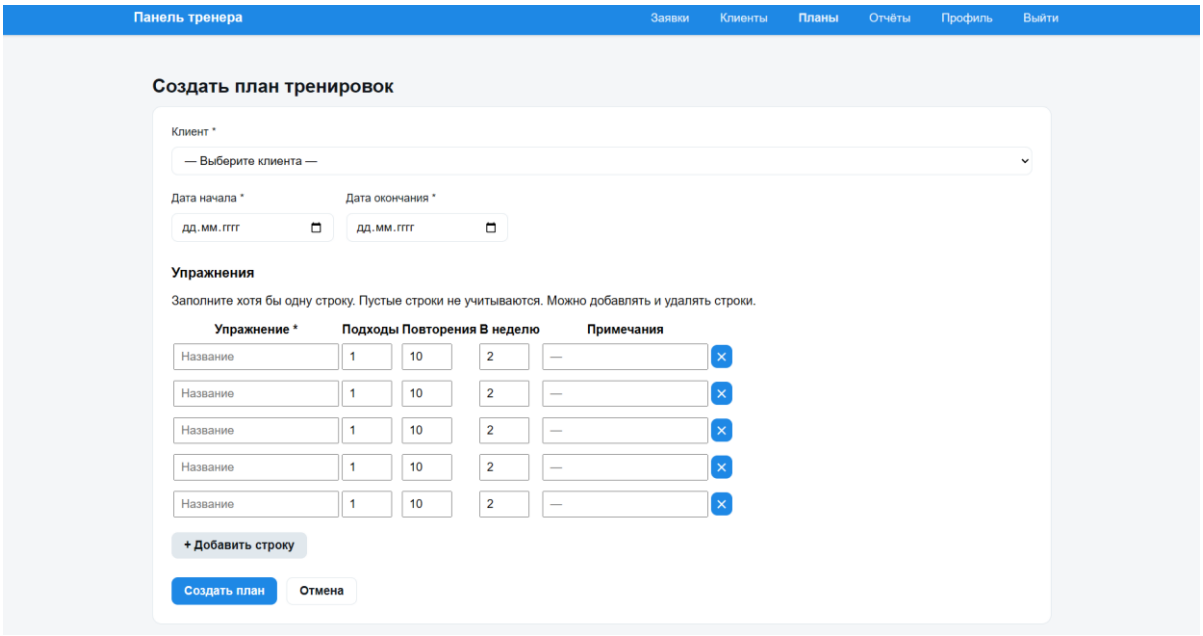


Рисунок 6, Форма создания плана тренировок

5. Работа клиента с тренировками

Клиент после входа на /client/ видит главную панель. В разделе /client/trainers/ доступен каталог тренеров. Нажав на кнопку подачи заявки (/client/trainers/<id>/apply/), клиент заполняет цель тренировок и отправляет POST-запрос.

Статус своих заявок клиент отслеживает в /client/applications/. После принятия заявки тренером клиенту становится доступен план тренировок (/client/plan/) и сегодняшние упражнения (/client/today/). На странице сегодняшних упражнений для каждого упражнения отображается чекбокс и поле для ввода пульса. Отметка выполнения отправляется через POST /client/today/exercise/<id>/complete/ с указанием пульса. Упражнение засчитывается только при наличии пульса.

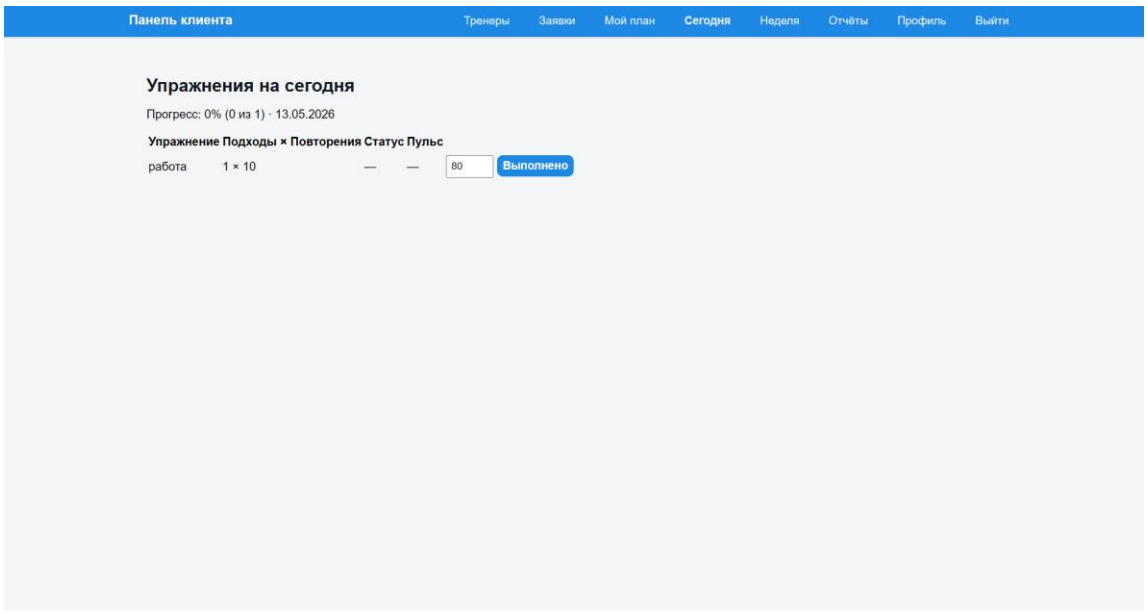


Рисунок 7, Страница сегодняшних упражнений

6. Отчёты

Тренер в разделе /trainer/reports/ может скачать отчёты: список клиентов (/trainer/reports/clients/), расписание (/trainer/reports/schedule/) и план конкретного клиента (/trainer/reports/plan/client/<client_id>/). Формат файла выбирается через параметр ?format=docx или ?format=xlsx (по умолчанию docx).

Клиент в разделе /client/reports/ скачивает свой план на сегодня (/client/reports/today/) и недельное расписание (/client/reports/weekly/) с выбором формата (см. рис. 8)

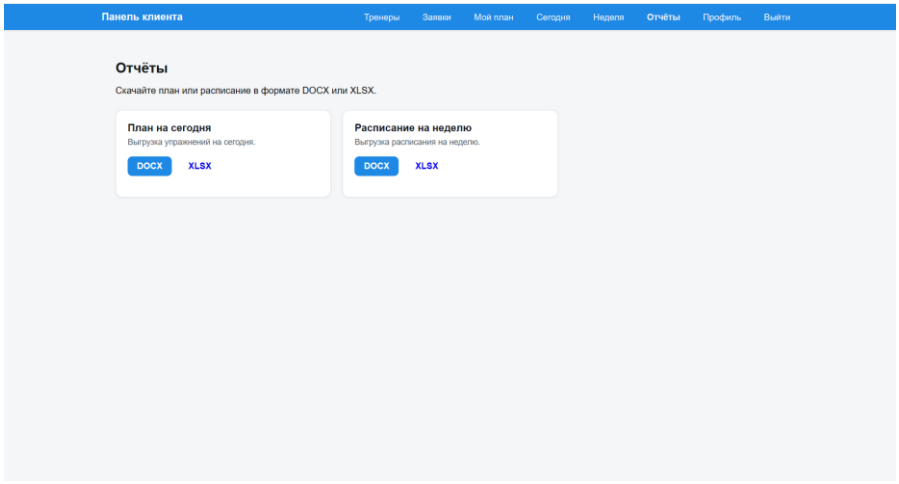


Рисунок 8, Страница скачивания плана тренировок

7. Просмотр профиля и выход

Любой авторизованный пользователь может просмотреть свой профиль по адресу GET /profile/ (данные получаются из GET /api/v1/profile/). Для выхода из системы используется GET /logout/, который сбрасывает сессию.

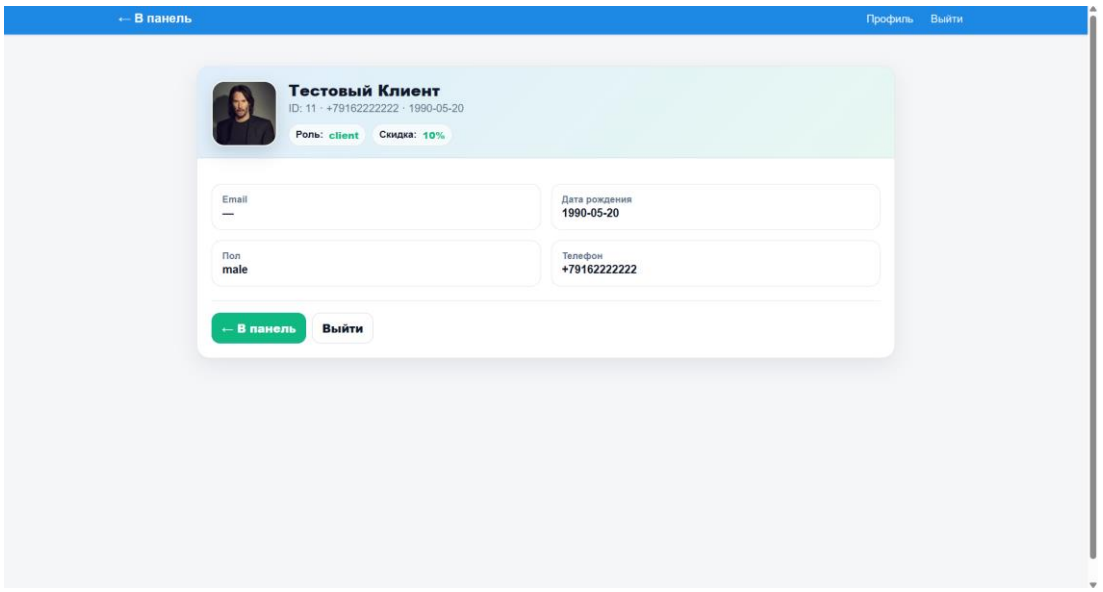


Рисунок 9, Профиль клиента

3.5.3 Руководство по развертыванию приложения

Требования к системе:

Установленный Python 3.8+.

PostgreSQL 17+.

Git (опционально).

Пошаговая инструкция:

Шаг 1. Установка PostgreSQL и создание базы данных

Скачайте и установите PostgreSQL с официального сайта. В процессе установки задайте пароль суперпользователя (например, postgres). Создайте базу данных fitness_db через pgAdmin (см. рис. 10) или командную строку:

CREATE DATABASE fitness_db;

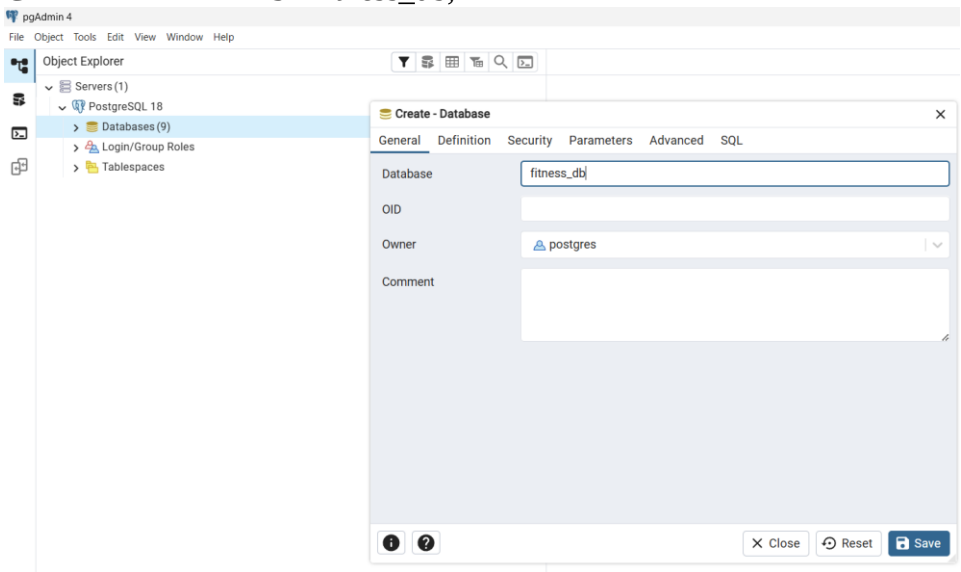


Рисунок 10, Процесс создания базы данных в pgAdmin

Шаг 2. Клонирование репозитория (опционально)

Если проект получен не через Git, этот шаг можно пропустить.
git clone https://github.com/ZiryanovAlan/fitness
cd fitness_zyranov_cr

Шаг 3. Запуск FastAPI (API-бэкенд)

FastAPI необходимо запускать первым, так как Django-приложение будет обращаться к его эндпоинтам.

3.1. Переход в папку API
cd fintes_zyranov2
3.2. Создание и активация виртуального окружения
python -m venv .venv
3.3. Установка зависимостей
pip install -r requirements.txt
3.4. Настройка переменных окружения
Создайте в папке fintes_zyranov2/ файл .env со следующим содержимым:
DB_USER=postgres
DB_PASSWORD=ваш_пароль
DB_HOST=localhost
DB_PORT=5432
DB_NAME=fitness_db
JWT_SECRET_KEY=ваш_секретный_ключ
ACCESS_TOKEN_EXPIRE_MINUTES=60

API_HOST=127.0.0.1
API_PORT=8001
API_RELOAD=True
3.5. Запуск FastAPI-сервера
python app/main.py

После запуска документация API будет доступна:
Swagger UI: http://127.0.0.1:8001/docs
ReDoc: http://127.0.0.1:8001/redoc
Базовый URL (префикс роутов): http://127.0.0.1:8001/api/v1

Шаг 4. Запуск Django (веб-интерфейс)

Django-часть по умолчанию обращается к FastAPI по адресу http://127.0.0.1:8001/api/v1. Если FastAPI запущен на другом хосте/порту, задайте переменную окружения FASTAPI_URL в настройках Django.

4.1. Переход в папку Django
Откройте новый терминал (второй) и выполните:
cd fitness_django
4.2. Создание и активация виртуального окружения
python -m venv .venv

4.3. Установка зависимостей
pip install -r requirements.txt

4.4. Применение миграций Django
Миграции нужны только для служебных таблиц (сессии, административная панель):
python manage.py migrate

4.5. Запуск Django-сервера
python manage.py runserver

Веб-приложение станет доступно по адресу http://127.0.0.1:8000/.

Шаг 5. Проверка работоспособности
5.1. Убедитесь, что оба сервера (FastAPI на порту 8001 и Django на порту 8000) запущены.
5.2. Откройте браузер и перейдите на http://127.0.0.1:8000.
5.3. Выполните вход под одним из тестовых пользователей (см. таблицу 12 в разделе 3.5.2).
5.4. Убедитесь, что после входа открывается дашборд, соответствующий роли, и данные успешно загружаются из API.

Примечания
При получении от API статуса 401 (неавторизован) Django-приложение автоматически сбрасывает сессию и перенаправляет пользователя на страницу входа.
Все сообщения об ошибках и успешных операциях выводятся через django.contrib.messages.
Для остановки серверов используйте сочетание Ctrl + C в каждом терминале.

3.5. Отчет о тестировании проекта

3.5.1. Информационный заголовок

Таблица 11, Информационный заголовок

Параметр	**Значение**
ID проекта	ФИТНЕС-АРМ-2026
Версия сборки	v1.0.0 (Django + FastAPI v1)
ФИО тестировщика	Зырянов Алан Русланович
Тип отчета	Итоговый отчет о тестировании
Дата составления	13 мая 2026 г.

3.5.2 Резюме

В ходе тестирования выполнена проверка всех функциональных требований технического задания.

Протестированы ролевые сценарии пользователей (клиент, тренер, менеджер, администратор), работа REST API на FastAPI и веб-интерфейса на Django.

В рамках проекта реализовано 156 автоматизированных тестов, включая:

- 84 API-тестов (test_auth, test_profile, test_admin, test_managers, test_trainers, test_clients);
- 199 тестов Django (представления, маршрутизация, обработка ошибок, работа с сессией, ui-тесты);

Все тесты завершены успешно, критические и блокирующие дефекты не выявлены.

Покрытие API-части проекта составляет 93%, покрытие Django-контроллеров и представлений - 75%. Подробнее см. в Приложении А.

Система соответствует требованиям технического задания и может быть рекомендована к опытной эксплуатации.

3.5.3 Область тестирования

Таблица 12 – Модули тестирования

№	**Модуль**	**Описание**
1	Аутентификация (/auth)	Вход, JWT-токены, выход, проверка ошибок авторизации и токенов.
2	Профиль пользователя (/profile)	Получение профиля, проверка ролей и доступа без токена.
3	Администратор (/admin)	CRUD-операции с тренерами, проверка прав доступа и валидации.
4	Менеджер (/managers)	CRUD-операции с клиентами, изменение скидок, проверка ограничений.
5	Тренер (/trainers)	Работа с заявками, тренировочными планами и отчетами.
6	Клиент (/clients)	Создание заявок, просмотр расписания и отметка выполнения упражнений.
7	Интеграционные сценарии	Полный цикл взаимодействия между ролями и API.
8	Веб-интерфейс (Django)	Проверка форм, сессий, сообщений и редиректов.
9	UI-сценарии (Selenium)	Проверка авторизации и поведения интерфейса.

3.5.4 Метрики и статистика

Таблица 13 – Количественные показатели тестирования

Показатель	**Значение**
Всего разработано API-тестов	84
Всего разработано тестов Django	199
Общее количество тест-кейсов	283
Количество пройденных тестов	283
Количество не пройденных тестов	0

Количество выявленных дефектов	0
Процент успешно пройденных тестов	100%

Таблица 14 – Покрытие кода

Компонент	**Инструмент**	**Покрытие**
API v1 (FastAPI)	pytest-cov	93%
Контроллеры Django	pytest-cov	75%

Для ознакомления с результатами тестирования и покрытия кода наглядно см. Приложении А, рисунки 11-12.

3.5.5 Примеры тест-кейсов

Для наглядности представлено несколько примеров тест-кейсов в таблице 15.

Таблица 15 – Примеры тест-кейсов

Идентификатор	**Модуль**	**Заголовок**	**Ожидаемый результат**	**Фактический результат**
AUTH_01	Аутентификация	Успешный вход администратора	Статус 200, токен	☑ Пройден
AUTH_02	Аутентификация	Неверный пароль	Статус 401	☑ Пройден
ADMIN_04	Администратор	Создание тренера с коротким паролем	Статус 422	☑ Пройден
MANAGER_05	Менеджер	Обновление скидки клиента	Статус 200	☑ Пройден
TRAINER_07	Тренер	Отклонение заявки без причины	Статус 400	☑ Пройден
CLIENT_09	Клиент	Создание заявки без цели	Статус 400	☑ Пройден
INT_12	Интеграция	Полный цикл работы	Все этапы завершены	☑ Пройден
UI_13	Веб-интерфейс	Ошибка входа	Сообщение об ошибке	☑ Пройден

4. ГЛОССАРИЙ

4.1 Сокращения

Таблица 16 – Список сокращений

Сокращение	Расшифровка
API	Application Programming Interface – интерфейс программирования приложений
АРМ	Автоматизированное рабочее место
БД	База данных
СУБД	Система управления базами данных
JWT	JSON Web Token – стандарт для создания токенов доступа
ORM	Object-Relational Mapping – объектно-реляционное отображение
RBAC	Role-Based Access Control – управление доступом на основе ролей
CRUD	Create, Read, Update, Delete – создание, чтение, обновление, удаление
UML	Unified Modeling Language – унифицированный язык моделирования
ERD	Entity-Relationship Diagram – диаграмма «сущность-связь»
JSON	JavaScript Object Notation – текстовый формат обмена данными
HTTP	HyperText Transfer Protocol – протокол передачи гипертекста
REST	Representational State Transfer – архитектурный стиль для веб-API
JOSE	Javascript Object Signing and Encryption – стандарты для подписи и шифрования JSON
ASGI	Asynchronous Server Gateway Interface – асинхронный интерфейс шлюза сервера

UI	User Interface – пользовательский интерфейс

Продолжение таблицы 9

Сокращение	Расшифрока
HTML	HyperText Markup Language – язык разметки гипертекста
CSS	Cascading Style Sheets – каскадные таблицы стилей

4.2 Термины

Таблица 17 – Список терминов

Термин	Определение
Аутентификация	Процесс проверки подлинности пользователя (например, по паре логин/пароль).
Авторизация	Процесс предоставления доступа к ресурсам после успешной аутентификации на основе роли.
Access token	Короткоживущий JWT-токен, подтверждающий личность пользователя и его права.
Refresh token	Долгоживущий JWT-токен, используемый для получения новой пары токенов без повторного ввода пароля.
Перевозчик	Организация или индивидуальный предприниматель, владеющие транспортными средствами и обслуживающие маршруты общественного транспорта.
Маршрут	Установленный путь следования транспортного средства между начальной и конечной остановками, имеющий уникальный номер, тип транспорта и расписание.
Остановка	Место на маршруте, оборудованное для посадки и высадки пассажиров. Характеризуется названием, адресом и наличием павильона.
Рейс	Конкретное отправление транспортного средства по определённому маршруту в заданные дату и время.
Статус рейса	Состояние рейса в ходе его выполнения: запланирован, отправлен, задержан, отменён, завершён.
Табель рабочего времени	Документ, фиксирующий выходы на работу сотрудников (водителей и кондукторов) с указанием количества отработанных дней и часов.

Группа	Объединение пользователей (операторов и обычных пользователей) вокруг одного администратора. Позволяет администратору управлять только «своими» пользователями.

Продолжение таблицы 17

Термин	Определение
Pydantic	Библиотека для валидации данных и создания схем на Python, встроенная в FastAPI.
SQLModel	ORM, построенный на основе SQLAlchemy и Pydantic, позволяющий использовать одни модели для работы с БД и валидации.
Alembic	Инструмент для управления миграциями схемы базы данных в проектах, использующих SQLAlchemy / SQLModel.
Миграции	Изменение схемы базы данных (добавление/удаление таблиц, полей, индексов) с сохранением данных.
Фикстура(тестирование)	Функция в pytest, предоставляющая тестовое окружение (клиент, сессию БД, тестового пользователя) и автоматически управляющая его ресурсами.
Покрытие кода	Метрика, показывающая, какая часть исходного кода выполнена во время прогона тестов. Измеряется в процентах.
Uvicorn	это высокопроизводительный ASGI-сервер для Python, предназначенный для работы с асинхронными веб-приложениями.
pgAdmin	программное обеспечение для управления базами данных PostgreSQL с открытым исходным кодом.
FastAPI	Веб-фреймворк для создания веб-API на языке Python, ориентированный на высокую производительность и простоту использования.
Swagger	Спецификация для машиночитаемого описания HTTP API, таких как REST-API.
OpenPyXL	Библиотека Python для работы с файлами Excel.
Xlsxwriter	Модуль Python для создания файлов в формате Excel 2007+ XLSX.

Pandas	Программная библиотека на языке Python для обработки и анализа данных, в частности структурированных данных, размещённых в таблицах.
JWT-токен	Компактный URL-безопасный способ представления утверждений (claims) между двумя сторонами. Используется для передачи информации о пользователе и его роли.

Продолжение таблицы 17

Термин	Определение
SPA	Тип веб-приложения, которое загружает одну HTML-страницу и динамически обновляет её содержимое без полной перезагрузки при взаимодействии с пользователем.
DRF	Библиотека для Python, предназначенная для создания RESTful API (интерфейсов прикладного программирования).
Excel	Программа для работы с электронными таблицами, разработанная компанией Microsoft.

5. СПИСОК ИСТОЧНИКОВ

1. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Процессы жизненного цикла программных средств. – М.: Стандартинформ, 2011. – 96 с.
2. ГОСТ 19.401-78. Единая система программной документации. Текст программы. Требования к содержанию и оформлению. – М.: Издательство стандартов, 1986. – 4 с.
3. FastAPI Documentation [Электронный ресурс] – Режим доступа: https://fastapi.tiangolo.com (дата обращения: 10.05.2026).
4. SQLModel Documentation [Электронный ресурс] – Режим доступа: https://sqlmodel.tiangolo.com (дата обращения: 10.05.2026).
5. PostgreSQL Documentation [Электронный ресурс] – Режим доступа: https://www.postgresql.org/docs/ (дата обращения: 10.05.2026).
6. Django Documentation [Электронный ресурс] – Режим доступа: https://docs.djangoproject.com/ (дата обращения: 10.05.2026).
7. Pydantic Documentation [Электронный ресурс] – Режим доступа: https://docs.pydantic.dev (дата обращения: 10.05.2026).
8. OpenPyXL Documentation [Электронный ресурс] – Режим доступа: https://openpyxl.readthedocs.io (дата обращения: 10.05.2026).
9. pytest Documentation [Электронный ресурс] – Режим доступа: https://docs.pytest.org (дата обращения: 10.05.2026).
10. Selenium WebDriver [Электронный ресурс] – Режим доступа: https://www.selenium.dev/documentation/ (дата обращения: 10.05.2026).
11. Fowler M. Patterns of Enterprise Application Architecture. – Addison-Wesley, 2002. – 560 с. (рус. пер.: Фаулер М. Архитектура корпоративных программных приложений. – М.: Вильямс, 2006. – 544 с.)
12. Martin R.C. Clean Code: A Handbook of Agile Software Craftsmanship. – Prentice Hall, 2009. – 464 с. (рус. пер.: Мартин Р. Чистый код. – СПб.: Питер, 2019. – 464 с.)
13. JWT Introduction [Электронный ресурс] – Режим доступа: https://jwt.io/introduction (дата обращения: 10.05.2026).
14. OAuth 2.0 Specification [Электронный ресурс] – Режим доступа: https://oauth.net/2/ (дата обращения: 10.05.2026).
15. REST API Best Practices [Электронный ресурс] – Режим доступа: https://restfulapi.net (дата обращения: 10.05.2026).
16. Argon2: The Memory-Hard Function [Электронный ресурс] – Режим доступа: https://github.com/P-H-C/phc-winner-argon2 (дата обращения: 10.05.2026).
17. Alembic: Database Migrations [Электронный ресурс] – Режим доступа: https://alembic.sqlalchemy.org (дата обращения: 10.05.2026).
18. Uvicorn ASGI Server [Электронный ресурс] – Режим доступа: https://www.uvicorn.org (дата обращения: 10.05.2026).
19. Swagger UI Documentation [Электронный ресурс] – Режим доступа: https://swagger.io/tools/swagger-ui/ (дата обращения: 10.05.2026).

20. Metanit: Python и FastAPI [Электронный ресурс] – Режим доступа: https://metanit.com/python/fastapi/ (дата обращения: 10.05.2026).
21. Грибова А.В. Лекции по дисциплине «МДК 05.02 Разработка кода информационных систем». – Екатеринбург: ЕМК, 2026.
22. Грибова А.В. Лекции по дисциплине «МДК 05.03 Тестирование информационных систем». – Екатеринбург: ЕМК, 2026.

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была разработана и протестирована информационная система АРМ «Фитнес-центр», полностью соответствующая техническому заданию. Разработана серверная часть в виде REST API на базе FastAPI с использованием SQLModel, PostgreSQL и JWT-аутентификации. API имеет единственную версию v1, так как в ходе тестирования дефектов, требующих обратно несовместимых изменений, не выявлено; реализованы пагинация, фильтрация, ролевой доступ (RBAC), автоматическая документация Swagger/ReDoc. Разработано веб-приложение на Django, которое выступает клиентом к API. Реализованы все сценарии для клиентов, тренеров, менеджеров и администраторов: выбор тренера, создание и обработка заявок, формирование тренировочных планов с упражнениями, отслеживание прогресса с фиксацией пульса, выгрузка отчётов (Word/Excel), а также CRUD-операции с клиентами и тренерами. Организовано тестовое окружение: написаны автоматизированные API и Django тесты (pytest), UI-тесты (Selenium), использованы фикстуры. Выполнено интеграционное тестирование, дефектов не выявлено. Подготовлена полная документация: ERD-модель, файловая структура, описание алгоритмов, руководство пользователя, руководство по развертыванию, глоссарий, список источников.

Направления дальнейшего развития:

- Добавление мобильного клиента с использованием существующего API.
- Внедрение уведомлений (email/SMS) о статусе заявок и тренировках.
- Расширение отчётности (анализ посещаемости, эффективности тренировок, подключение фитнес-трекеров).

Таким образом, цели курсового проекта достигнуты, результаты могут быть использованы как учебный пример интеграции Django и FastAPI, а также как основа для реального внедрения в небольших фитнес-центрах.

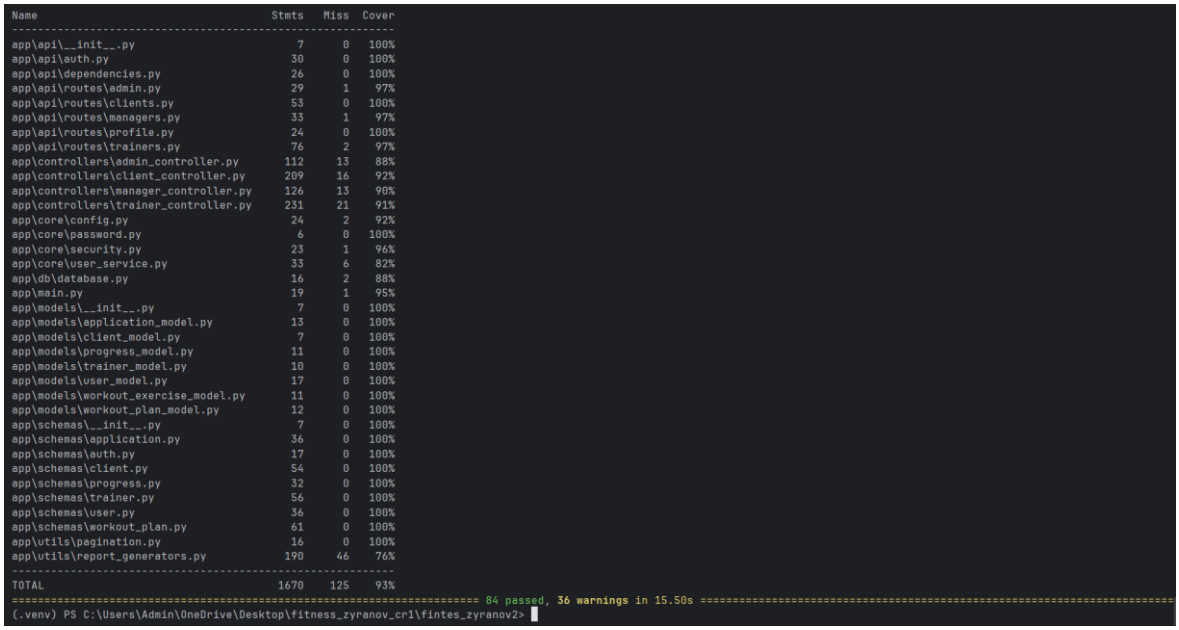


Рисунок 11, Результаты запуска и покрытие API-тестов

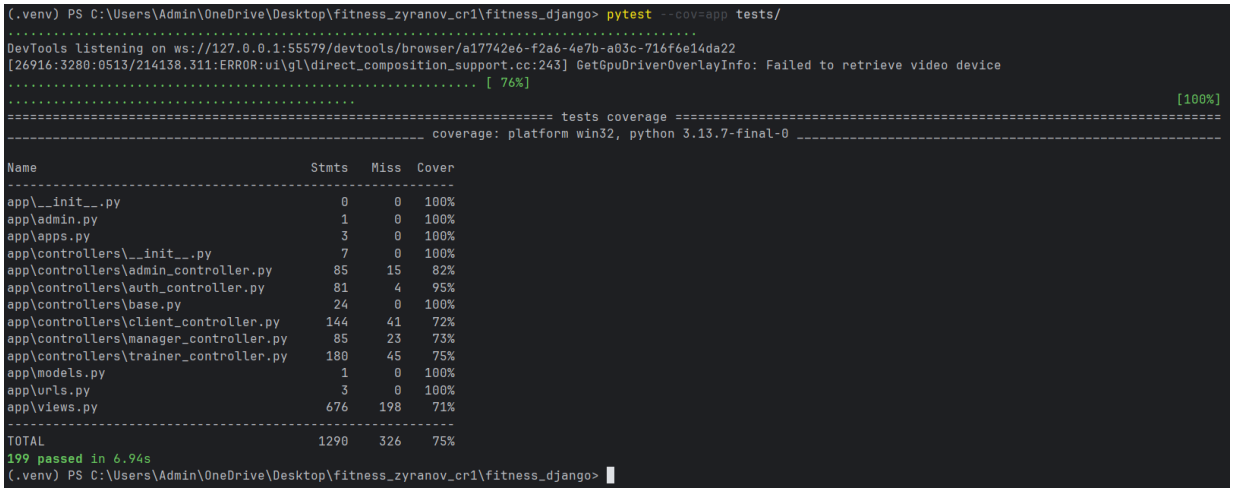


Рисунок 12, Результаты запуска и покрытие тестов Django

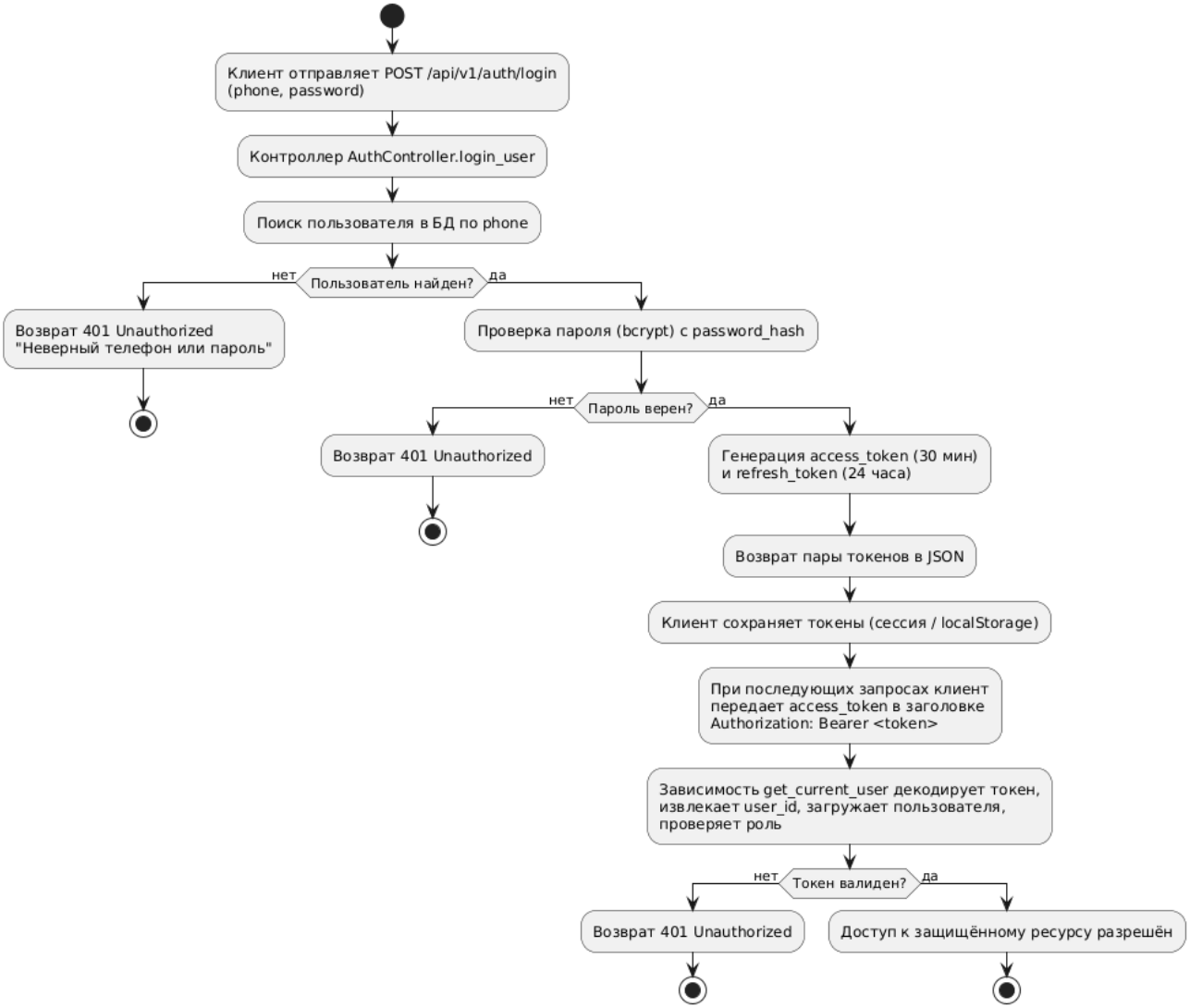


Рисунок 13, Диаграмма алгоритма Jwt

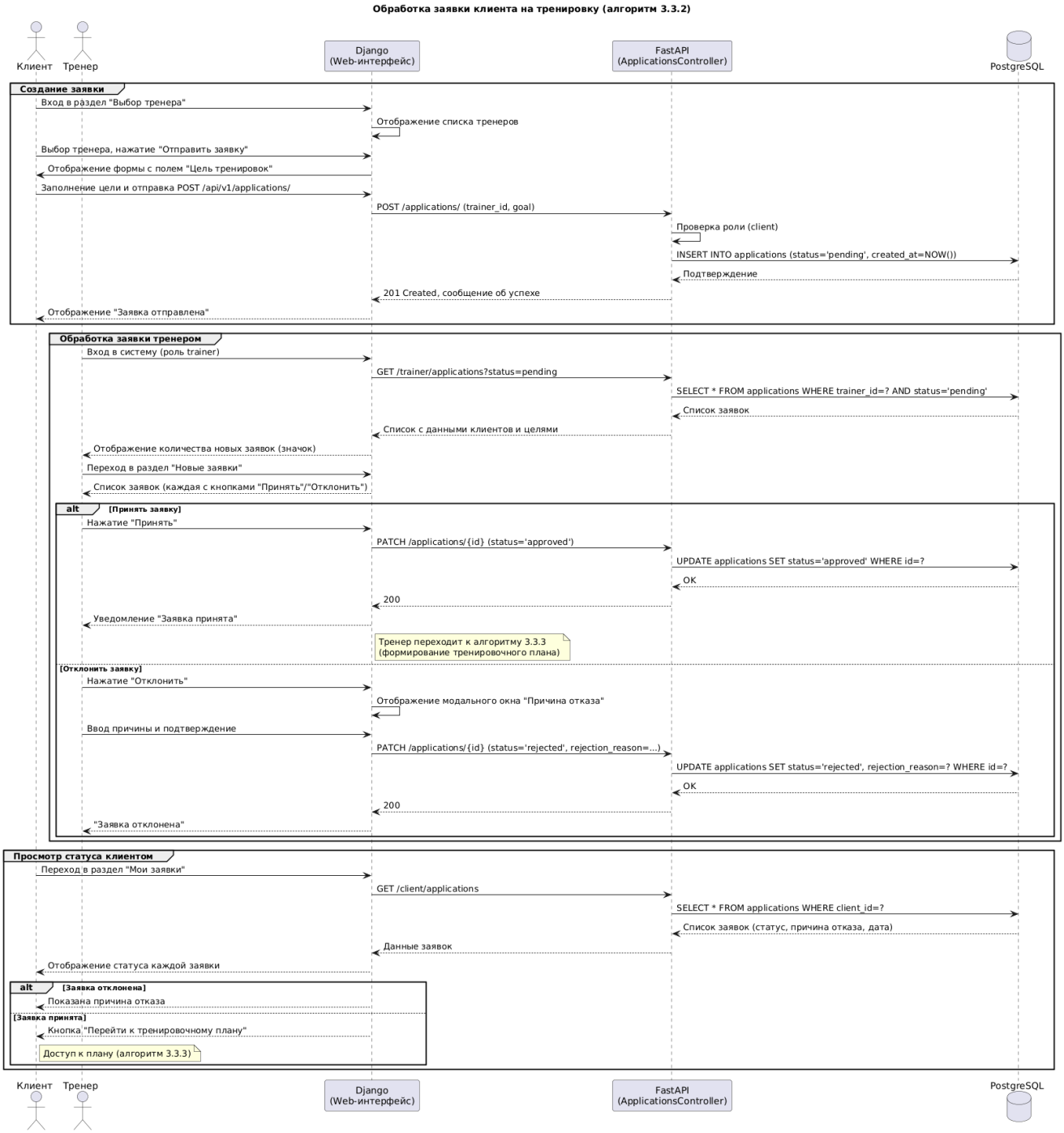


Рисунок 14, Алгоритм обработки заявки клиента на тренировку